Dipartimento di Informatica
Laboratorio di Architettura degli Elaboratori 1
(LAE1)



UNIVERSITÀ DEGLI STUDI
DI MILANO

Esercizi logica combinatoria, signal

Docenti del corso

Matteo Rè matteo.re@unimi.it
Massimo Walter Rivolta massimo.rivolta@unimi.it
Gabriella Trucco gabriella.trucco@unimi.it

signal

- E’ l'elemento fondamentale di **interconnessione** in VHDL.
- Modellizzazione di una **connessione fisica** o di un filo/nodo all'interno di un circuito.
- Le modifiche ai segnali **non sono immediate**, ma avvengono dopo un ritardo di propagazione (gestito dal simulatore).
- Le assegnazioni ai segnali utilizzano l'operatore **<=** (assegnazione concorrente, valutata "in parallelo").
- *Esempio 1:* Y <= A and B; (il valore di Y è ricalcolato ogni volta che A o B cambiano).
- Esempio 2:

```
X <= A and B;
Y <= C or D;
Z <= not X;
```

Le tre istruzioni vengono valutate contemporaneamente, non in sequenza. L’ordine con cui le scriviamo non è importante.

Signal

Natura fisica del segnale

- Filo elettrico: un segnale modella un **filo elettrico** o un **bus** che connette due o più punti logici del circuito.
- Ogni volta che si dichiara un segnale, il tool di sintesi lo interpreta come un percorso di instradamento potenziale nel circuito finale.
- I segnali sono l'unico mezzo per far comunicare le diverse parti concorrenti e i componenti (le entità istanziate) all'interno di un'architettura.

Signal

- Natura temporizzata: la caratteristica fondamentale del segnale è che le sue modifiche non avvengono mai in modo istantaneo.
- Assegnazione pianificata (pianificazione temporale):
  - Il valore in uscita non cambia immediatamente, ma viene schedulato per aggiornarsi.
- Nelle simulazioni ideali, l'aggiornamento **non è istantaneo**.
  - **Fase 1:** il simulatore calcola il valore (a tempo zero).
  - **Fase 2:** il valore viene scritto sul filo.

Esempio 1

Delta-cycle: passo temporale virtuale a tempo zero usato dal simulatore per rispecchiare il ritardo di propagazione fisico dell'hardware reale.

```
Y <= A and B;
Z <= C;
```

Istruzioni indipendenti: Y e Z viaggiano su strade separate e non dipendono l'uno dall'altra.

Valutazione e aggiornamento in UN UNICO delta-cycle.

- **Fase 1 (valutazione):** il simulatore calcola in parallelo sia Y che Z.
- **Fase 2 (aggiornamento):** il simulatore aggiorna contemporaneamente sia il filo di Y che il filo di Z.

Totale: **1 solo delta-cycle** ed entrambi i segnali sono aggiornati e pronti.

Esempio 2

Y <= A and B;

Z <= not Y;

- **Primo delta-cycle:**
  - *Fase 1 (valutazione):* si calcola il nuovo Y (e si valuta Z usando il vecchio Y).
  - *Fase 2 (aggiornamento):* il filo di Y prende il nuovo valore.
- **Secondo delta-cycle:**
  - *Fase 1 (valutazione):* ora che Y è cambiato sul filo, il simulatore si accorge che Z deve essere ricalcolato usando il *nuovo* Y.
  - *Fase 2 (aggiornamento):* il filo di Z si aggiorna con il valore corretto.

Totale: **2 delta-cycle**.

Inizializzazione di un segnale nei testbench

- Quando scriviamo un testbench per simulare il circuito, i segnali in ingresso partono per default nello stato non inizializzato **'U'** (Undefined/Uninitialized).
- Se un ingresso è 'U', qualsiasi operazione logica (es. A and B) restituisce a sua volta 'U', bloccando la simulazione.
- Soluzione: usiamo l'operatore di assegnazione iniziale **:=** nella dichiarazione del segnale per forzare un avvio pulito

```
signal A : std_logic := '0'; -- Parte già a 0 invece di 'U'
```

Tipi di dato fondamentali

std_logic_vector

- Un array di std_logic.
- Utilizzato per rappresentare bus di dati, registri, ecc.

- Sintassi per la dichiarazione: (1)

```
segnale_bus : std_logic_vector(NUM_BIT-1 downto 0);
```

(2)

```
segnale_bus_2 : std_logic_vector(0 to NUM_BIT-1);
```

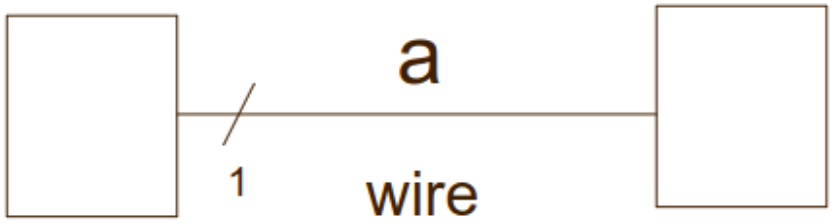
  - (1): da bit più significativo a meno significativo
  - (2): da bit meno significativo a più significativo

- **Esempio:** un bus a 8 bit:

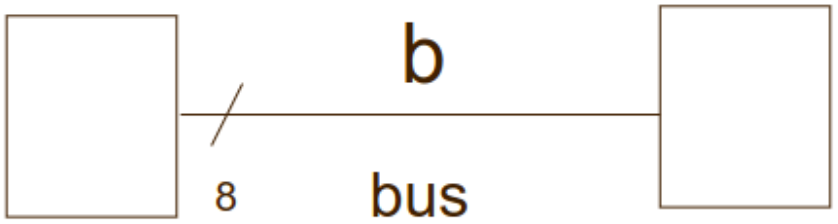
```
data_in : in std_logic_vector(7 downto 0);
```

Modellare fili e bus: segnali

```
SIGNAL  a : STD_LOGIC;
```




```
SIGNAL b : STD_LOGIC_VECTOR(7 DOWNTO 0);
```



Modellare fili e bus: segnali

```
SIGNAL a: STD_LOGIC;
SIGNAL b: STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL c: STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL d: STD_LOGIC_VECTOR(15 DOWNTO 0);
SIGNAL e: STD_LOGIC_VECTOR(8 DOWNTO 0);
                                .........
a <= ' 1' ;
b <= "0000";          -- Binary base assumed by default
c <= B"0000";         -- Binary base explicitly specified
d <= X"AF67";         -- Hexadecimal base
e <= O"723";          -- Octal base
```

Concatenare vettori

```
SIGNAL a: STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL b: STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL c, d, e: STD_LOGIC_VECTOR(7 DOWNTO 0);

a <= "0000";
b <= "1111";
c <= a & b;                    -- c = "00001111"

d <= '0' & "0001111";    -- d <= "00001111"

e <= '0' & '0' & '0' & '0' & '1' & '1' &
     '1' & '1';
                               -- e <= "00001111"
```

Operazione logica bit-wise (bit a bit)

Quando un operatore logico (and, or, xor, not, ecc...) viene applicato a due vettori dello stesso tipo e della stessa dimensione, l'operazione viene eseguita **bit-wise (bit a bit)**, ovvero:

- l'operatore viene applicato tra il bit *i* del primo vettore e il bit *i* del secondo vettore;
- il risultato del bit *i* di output dipende **solo** dal bit *i* di input;
- le singole operazioni sui bit sono **indipendenti** e avvengono in **parallelo** (contemporaneamente) nell'hardware.

Operatori logici su vettori (std_logic_vector)

Esempio: operazione AND tra due vettori a 4 bit.

Indice Bit	3	2	1	0
Data_A	0	1	0	1
Data_B	0	0	1	1
Operatore AND	↓ AND ↓	↓ AND ↓	↓ AND ↓	↓ AND ↓
Result_Y	0	0	0	1

L'assegnazione singola *Result_Y* <= *Data_A* and *Data_B*; è sintetizzata dal tool come **quattro porte AND separate**, ciascuna connettendo i bit allineati.

```
signal Data_A : std_logic_vector(3 downto 0) := "0101";
signal Data_B : std_logic_vector(3 downto 0) := "0011";
signal Result_Y : std_logic_vector(3 downto 0);
-- ...
Result_Y <= Data_A and Data_B; -- Risultato: "0001"
```

Bit	Porte Logiche Reali
Bit 3	$\text{Result_Y}(3) \leftarrow \text{Data_A}(3)\ \text{AND}\ \text{Data_B}(3)$
Bit 2	$\text{Result_Y}(2) \leftarrow \text{Data_A}(2)\ \text{AND}\ \text{Data_B}(2)$
Bit 1	$\text{Result_Y}(1) \leftarrow \text{Data_A}(1)\ \text{AND}\ \text{Data_B}(1)$
Bit 0	$\text{Result_Y}(0) \leftarrow \text{Data_A}(0)\ \text{AND}\ \text{Data_B}(0)$

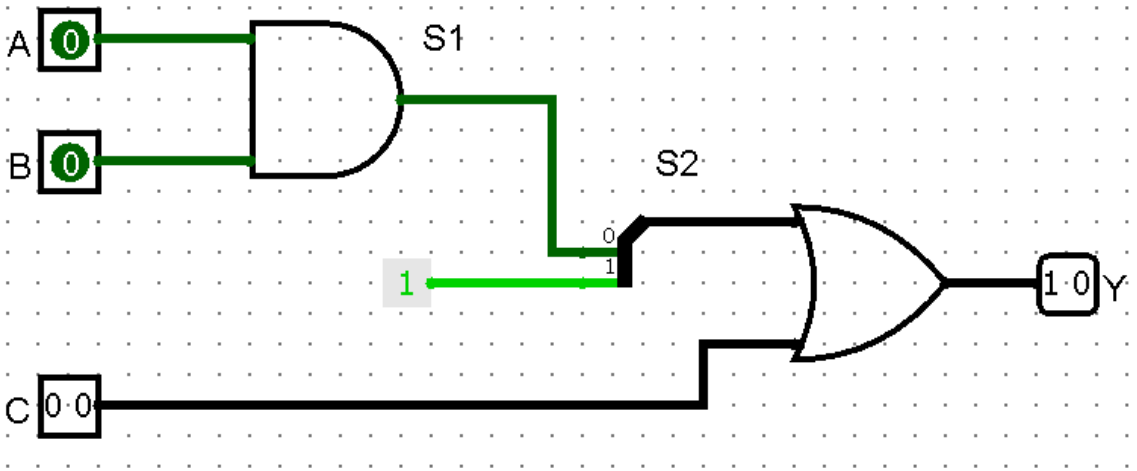
Altri operatori logici bit-wise

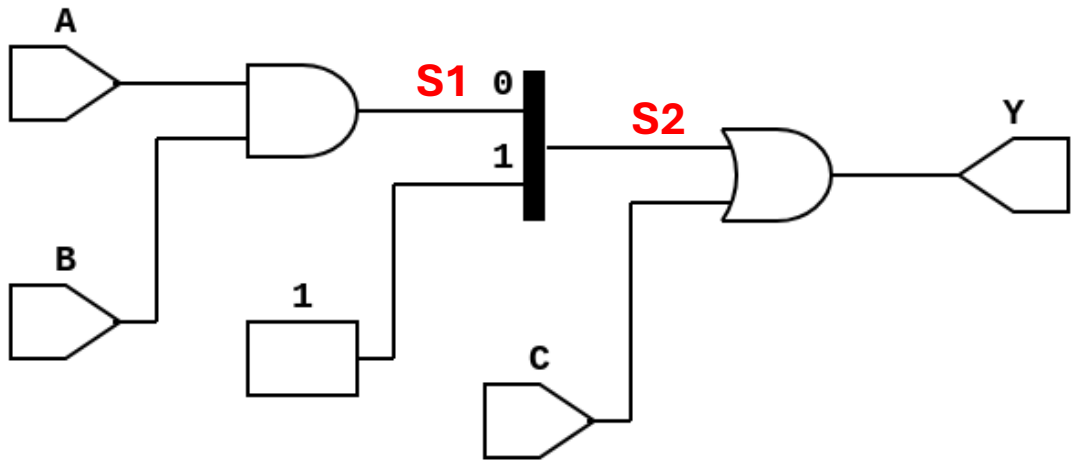
Lo stesso principio bit-wise si applica a tutti gli operatori logici standard in VHDL:

- **OR:** Bus_C <= Bus_A or Bus_B; (crea N porte OR).
- **XOR:** Bus_D <= Bus_A xor Bus_B; (crea N porte XOR, spesso utilizzate per **comparatori** bit-wise o generatori di parità).
- **NOT:** Bus_E <= not Bus_A; (crea N porte NOT, applicate a ogni bit del vettore).

Esercizio 1 – signal

Costruire in VHDL e simulare in ghdl/gtkwave il seguente circuito:



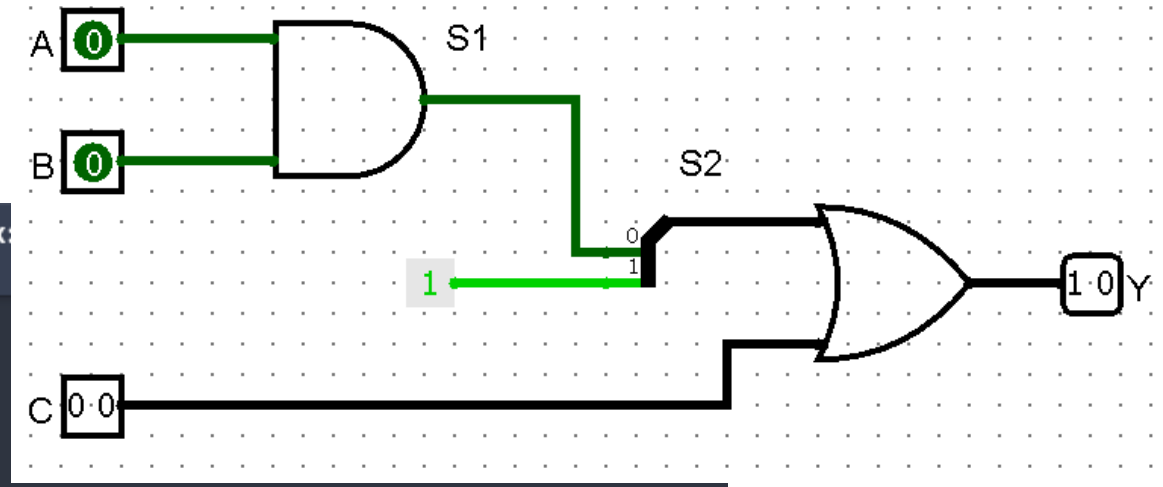


Chiamare l'entity «signals»

Riempire il testbench tb_signals.vhdl presente in LEZ2/ESE1 con i tutti i casi necessari

Creare esplicitamente i segnali interni necessari e fare attenzione ai bus

Esercizio 1 – signal.vhdl




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity signals is
        port(A, B: in std_logic; C: in std_logic_vector(1 downto 0);
                Y: out std_logic_vector(1 downto 0));
end;

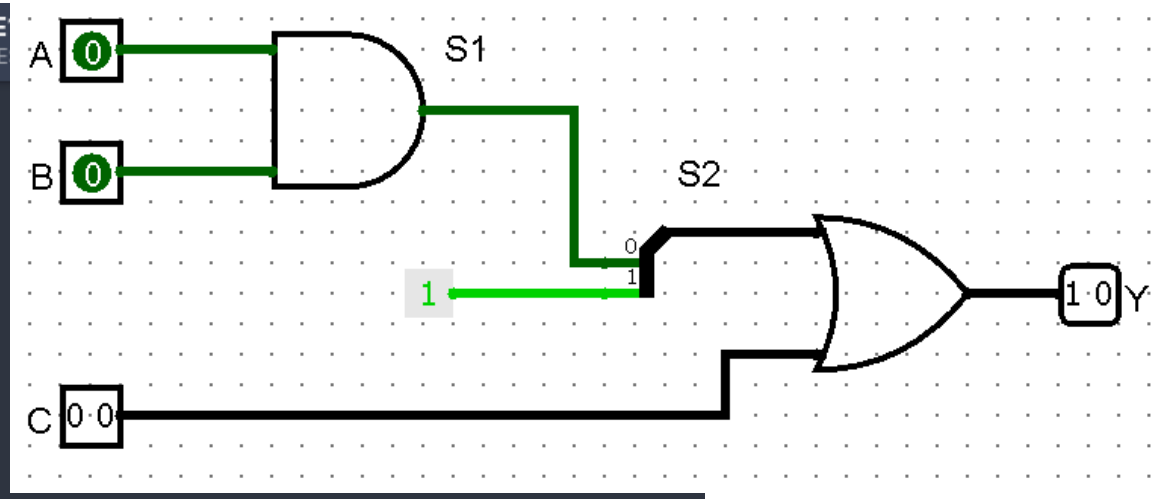
architecture arch_signals of signals is
        signal S1: std_logic;
        signal S2: std_logic_vector(1 downto 0);
begin
        S1 <= A and B;
        S2 <= '1' & S1;
        Y <= S2 or C;
end;
```

Esercizio 1 – tb_signal.vhdl

```
stimulus_process : process
begin
    -- Test Cases here

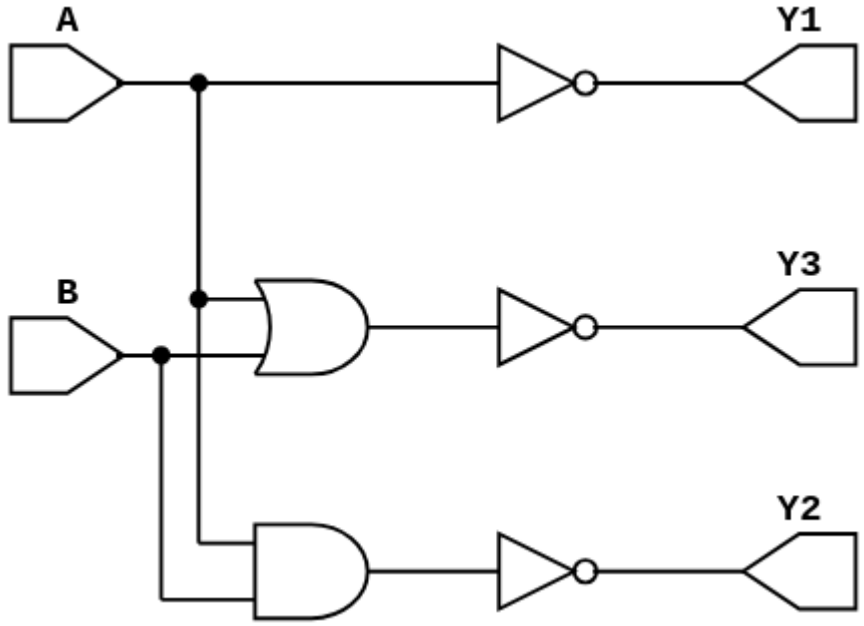
    A <= '0'; B <= '0'; C <= "00"; wait for 5 ns;
    A <= '0'; B <= '0'; C <= "01"; wait for 5 ns;
    A <= '0'; B <= '0'; C <= "10"; wait for 5 ns;
    A <= '0'; B <= '0'; C <= "11"; wait for 5 ns;
    A <= '0'; B <= '1'; C <= "00"; wait for 5 ns;
    A <= '0'; B <= '1'; C <= "01"; wait for 5 ns;
    A <= '0'; B <= '1'; C <= "10"; wait for 5 ns;
    A <= '0'; B <= '1'; C <= "11"; wait for 5 ns;
    A <= '1'; B <= '0'; C <= "00"; wait for 5 ns;
    A <= '1'; B <= '0'; C <= "01"; wait for 5 ns;
    A <= '1'; B <= '0'; C <= "10"; wait for 5 ns;
    A <= '1'; B <= '0'; C <= "11"; wait for 5 ns;
    A <= '1'; B <= '1'; C <= "00"; wait for 5 ns;
    A <= '1'; B <= '1'; C <= "01"; wait for 5 ns;
    A <= '1'; B <= '1'; C <= "10"; wait for 5 ns;
    A <= '1'; B <= '1'; C <= "11"; wait for 5 ns;

    wait; -- Mantiene il processo inattivo indefinitamente dopo l'ultimo stimolo
```



Esercizio 2

1) Creare l’entità NNN (Nand, Nor, Not) con ingressi A e B e uscite Y1=not A, Y2=A nand B, Y3=A nor B, definire le casistiche da testare nel testbench fornito e verificare che tutto funzioni. Salvare l’entity in LEZ2/ESE2 e usare il testbench fornito nella stessa cartella



OR + NOT = NOR

AND + NOT = NAND

Esercizio 3

2) Descrivere il funzionamento di un allarme casalingo così funzionante: tre ingressi A, B e C e due uscite Y1 e Y2, l'allarme per Y1 vale 1 solo se almeno uno tra A e B è 1, l'allarme per Y2 vale 1 solo se A e B sono uguali. Identificare le porte logiche per il problema e simulare l'intero circuito a partire dall'entity. Usare il testbench fornito in LEZ2/ESE3.

